

DESIGN DECISIONS DOCUMENT

TeamFlow – MERN Stack Project Management Application

1. Introduction

TeamFlow is a web-based Project Management System developed using the MERN (MongoDB, Express.js, React.js, and Node.js) technology stack. The application enables users to register securely, log in using JWT authentication, create projects, manage tasks, and track project progress through a responsive web interface.

The application follows a layered architecture that separates the frontend, backend, business logic, and database, making the system scalable, maintainable, and easy to extend with additional features in the future.

2. Technology Stack Selection

The MERN stack was selected because it enables full-stack JavaScript development. Using JavaScript across both frontend and backend simplifies development, reduces context switching, and allows code reuse.

Frontend

- React.js
- React Router
- Axios
- CSS

React provides reusable components, fast rendering through the Virtual DOM, and an excellent user experience.

Backend

- Node.js
- Express.js

Express simplifies REST API development by providing lightweight routing and middleware support.

Database

MongoDB Atlas

MongoDB is a NoSQL database that stores data as flexible JSON-like documents. It is suitable for applications where data structures may evolve over time.

3. Application Architecture

The project follows a layered architecture consisting of:

- Presentation Layer (React Frontend)

- API Layer (Express.js)
- Authentication Layer (JWT Middleware)
- Business Logic Layer (Controllers)
- Data Access Layer (Mongoose Models)
- Database Layer (MongoDB Atlas)

This separation improves code readability and makes testing and maintenance easier.

4. Authentication Strategy

JSON Web Token (JWT) authentication is used to secure protected API endpoints.

Reasons for choosing JWT

- Stateless authentication
- No server-side session storage
- Easy integration with React
- Suitable for REST APIs
- Scalable for cloud deployment

After successful login, the server generates a JWT token that is sent to the client. The client stores the token and includes it in the Authorization header for subsequent API requests. Protected routes verify the token before granting access.

5. Database Design

MongoDB Atlas was selected as the database because of its flexibility and cloud-hosted environment.

The application currently contains three main collections:

Users

Stores user information such as name, email, password, and role.

Projects

Stores project details including title, description, creator, and members.

Tasks

Stores task information including title, description, status, due date, and project reference.

Relationships between collections are maintained using ObjectId references through Mongoose.

6. REST API Design

The backend follows REST architectural principles.

Major API groups include:

Authentication APIs

- Register User
- Login User

Project APIs

- Create Project
- View Projects
- Update Project
- Delete Project

Task APIs

- Create Task
- View Tasks
- Update Task
- Delete Task
- View Tasks by Project

REST APIs make communication between the frontend and backend simple, predictable, and easy to test using tools like Postman.

7. Security Considerations

Several security measures were implemented during development.

Password Protection

User passwords are encrypted using bcrypt before being stored in MongoDB.

Authentication

JWT tokens ensure that only authenticated users can access protected resources.

Authorization

Protected routes verify user authentication before executing controller logic.

Environment Variables

Sensitive configuration values such as MongoDB connection strings and JWT secrets are stored in environment variables rather than hardcoded into the application.

8. Design Decisions

The following design decisions were made during development:

- Adopted the MVC (Model–View–Controller) architecture to separate concerns.
- Used React component-based development for better code reuse.

- Implemented Mongoose models to simplify database operations.
 - Used Axios for communication between React and Express APIs.
 - Designed RESTful endpoints for consistency and scalability.
 - Hosted the database on MongoDB Atlas for reliability and easy cloud deployment.
-

9. Future Enhancements

The current implementation can be extended with additional features such as:

- Real-time notifications using Socket.io
 - File upload support using Cloudinary
 - Email notifications using Nodemailer
 - Role-based access control with multiple user roles
 - Dashboard analytics and reporting
 - Search, filtering, and pagination
 - Activity logs and audit history
 - Docker containerization
 - CI/CD pipeline integration
 - Deployment on AWS, Azure, or Railway
-

10. Conclusion

TeamFlow demonstrates the implementation of a modern full-stack web application using the MERN stack. The system follows clean architectural principles, uses JWT-based authentication for security, and adopts RESTful API design for communication between the frontend and backend. The chosen technologies provide scalability, maintainability, and flexibility, making the application suitable for future enhancements and real-world project management scenarios.